Creating middleware with the `Use` extension, as you did in listings 31.3 and 31.4, is convenient, but it's not easy to test, and you're somewhat limited in what you can do. For example, you can't easily use DI to inject scoped services inside these basic middleware components. Normally, rather than call the `Use` extension directly, you'll encapsulate your middleware into a class that's functionally equivalent.

Custom middleware components don't have to derive from a specific base class or implement an interface, but they have a certain shape, as shown in listing 31.5. ASP.NET Core uses reflection to execute the method at runtime. Middleware classes should have a constructor that takes a `RequestDelegate` object, which represents the rest of the middleware pipeline, and they should have an `Invoke` function with a signature similar to

```
public Task Invoke(HttpContext context);
```

The `Invoke()` function is equivalent to the lambda function from the `Use` extension, and it is called when a request is received. The following listing shows how you could convert the headers middleware from listing 31.4 into a standalone middleware class.

Listing 31.5 Adding headers to a `Response` using a custom middleware component

```
public class HeadersMiddleware
{
    private readonly RequestDelegate _next;                    ❶
    public HeadersMiddleware(RequestDelegate next)             ❶
    {                                                          ❶
        _next = next;                                          ❶
    }                                                          ❶

    public async Task Invoke(HttpContext context)              ❷
```